CS315 · ALGORITHMS ANALYSIS & DESIGN

# Final Drill Solutions

One drill on Lectures 1 to 5 in the midterm format

**Muharib**

## 01 Solution Q1: MCQ

| # | Answer | Reason |
|---|---|---|
| 1 | **c** | A loop inside a loop → n × n |
| 2 | **b** | log n levels in the recursion tree, n work on each |
| 3 | **c** | n priority-queue operations, each log n |
| 4 | **a** | The element is the first one |
| 5 | **b** | Greedy fails on 0-1 but works for fractional |

## 02 Solution Q2: counting operations

i runs from 0 to n − 1, so the loop makes **n** iterations. The for header = 1 + (n + 1) = n + 2.

| Line | Worst | Best | Why |
|---|---|---|---|
| **c ← 0** | 1 | 1 | One assignment, once |
| **for i ← 0 to n − 1** | n + 2 | n + 2 | i ← 0 once, the test n + 1 times |
| **if A[i] > 0** | 2n | 2n | Index and comparison, n times |
| **c ← c + 1** | 2n | 0 | Addition and assignment. Worst: all positive. Best: none |
| **increment i** | 2n | 2n | Addition and assignment, n times |
| **return c** | 1 | 1 | |
| **Total** | **7n + 4** | **5n + 4** | Both are O(n) |

# 03 Solution Q3: sorting

## a) Insertion sort

| i | key | Array after the iteration |
|---|---|---|
| – | – | 4 7 1 8 3 5 |
| 2 | 7 | 4 7 1 8 3 5 |
| 3 | 1 | 1 4 7 8 3 5 |
| 4 | 8 | 1 4 7 8 3 5 |
| 5 | 3 | 1 3 4 7 8 5 |
| 6 | 5 | 1 3 4 5 7 8 |

## b) Merge sort

| | | | | |
|---|---|---|---|---|
| Divide | [6 3 8 1 5 2 7 4] | | | |
| | [6 3 8 1] | | [5 2 7 4] | |
| | [6 3] | [8 1] | [5 2] | [7 4] |
| | [6] [3] | [8] [1] | [5] [2] | [7] [4] |
| Merge | [3 6] | [1 8] | [2 5] | [4 7] |
| | [1 3 6 8] | | [2 4 5 7] | |
| | [1 2 3 4 5 6 7 8] | | | |

## c) Recursion tree

Levels = log(8) + 1 = 3 + 1 = 4 (levels 0 to 3)

Level 2 = $2^2$ = 4 problems, each of size $8 \div 2^2 = 2$

Total work = $6n(\log(n) + 1) = 6 \times 8 \times 4 = 192$

# 04 Solution Q4: hash table

| Key | 20 | 34 | 45 | 70 | 56 | 13 |
|---|---|---|---|---|---|---|
| k mod 11 | 9 | 1 | 1 | 4 | 1 | 2 |

## a) Linear probing

| Key | k mod 11 | Probes | Cell |
|---|---|---|---|
| **20** | 9 | 9 | **9** |
| **34** | 1 | 1 | **1** |
| **45** | 1 | 1 → 2 | **2** |
| **70** | 4 | 4 | **4** |
| **56** | 1 | 1 → 2 → 3 | **3** |
| **13** | 2 | 2 → 3 → 4 → 5 | **5** |

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| | 34 | 45 | 56 | 70 | 13 | | | | 20 | |

## b) Double hashing

Cell = (h(k) + j·d(k)) mod 11, with j = 0, 1, 2 ...

| Key | k mod 11 | 7 − (k mod 7) | Probes | Cell |
|---|---|---|---|---|
| **20** | 9 | 1 | 9 | **9** |
| **34** | 1 | 1 | 1 | **1** |
| **45** | 1 | 4 | 1 → 5 | **5** |
| **70** | 4 | 7 | 4 | **4** |
| **56** | 1 | 7 | 1 → 8 | **8** |
| **13** | 2 | 1 | 2 | **2** |

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| | 34 | 13 | | 70 | 45 | | | 56 | 20 | |

### c) Separate chaining

| Index | Chain |
|---|---|
| 0 | ø |
| 1 | 34 → 45 → 56 |
| 2 | 13 |
| 3 | ø |
| 4 | 70 |
| 5 | ø |
| 6 | ø |
| 7 | ø |
| 8 | ø |
| 9 | 20 |
| 10 | ø |

## 05 Solution Q5: LCS

If the two characters match: the diagonal + 1. Otherwise: the larger of the cell above and the

cell to the left.

| | ∅ | A | B | D | C | B |
|---|---|---|---|---|---|---|
| ∅ | 0 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 1 | 1 |
| C | 0 | 0 | 1 | 1 | 2 | 2 |
| A | 0 | 1 | 1 | 1 | 2 | 2 |
| D | 0 | 1 | 1 | 2 | 2 | 2 |
| B | 0 | 1 | 2 | 2 | 2 | 3 |

LCS length = **3**. LCS: ADB, BCB, BDB (any one is correct).

## 06 Solution Q6: matrix chain

Dimensions: p = (6, 4, 10, 2, 8). Each cell = the smallest

`m[i,k] + m[k+1,j] + $p_{i-1} \cdot p_k \cdot p_j$`.

```
m[A,B] (k = A): 0 + 0 + 6·4·10 = 240
```

```
m[B,C] (k = B): 0 + 0 + 4·10·2 = 80
```

```
m[C,D] (k = C): 0 + 0 + 10·2·8 = 160
```

```
m[A,C] (k = A): 0 + 80 + 6·4·2 = 128
```

```
m[A,C] (k = B): 240 + 0 + 6·10·2 = 360
```

```
m[B,D] (k = B): 0 + 160 + 4·10·8 = 480
```

```
m[B,D] (k = C): 80 + 0 + 4·2·8 = 144
```

```
m[A,D] (k = A): 0 + 144 + 6·4·8 = 336
```

```
m[A,D] (k = B): 240 + 160 + 6·10·8 = 880
```

```
m[A,D] (k = C): 128 + 0 + 6·2·8 = 224
```

| | A | B | C | D |
|---|---|---|---|---|
| **A** | 0 | 240 | 128 | **224** |
| **B** | — | 0 | 80 | 144 |
| **C** | — | — | 0 | 160 |
| **D** | — | — | — | 0 |

Minimum = **224** scalar multiplications. Optimal order: **((A(BC))D)**

# 07 Solution Q7: greedy

## a) Fractional knapsack

| Item | w | v | v ÷ w |
|---|---|---|---|
| A | 5 | 30 | 6 |
| C | 3 | 18 | 6 |
| B | 10 | 40 | 4 |
| D | 8 | 24 | 3 |

1. Sort by value per kg, largest first.
2. Take all of A: weight 5, value 30.
3. Take all of C: weight 8, value 48.
4. 7 kg are left, so take 7 of the 10 kg of B: `7 ÷ 10 × 40 = 28`.

Total value = 30 + 18 + 28 = **76** (D is not taken).

## b) Activity selection

Sort by finish time, take the first activity, then every activity whose start ≥ the finish of the last one taken.

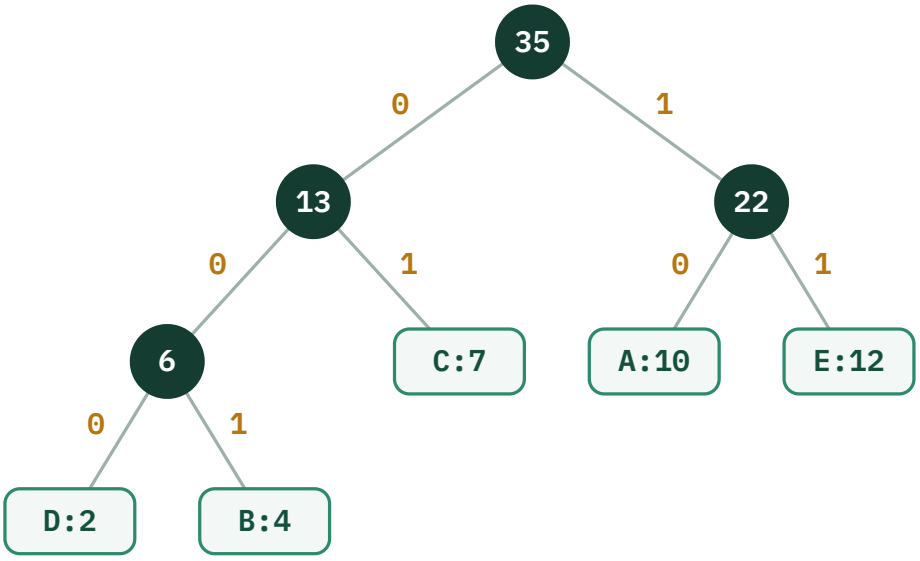
| Activity | s | f | Taken? | Why |
|---|---|---|---|---|
| a1 | 1 | 3 | ✓ | The first one |
| a2 | 2 | 5 | × | 2 < 3 |
| a3 | 4 | 6 | ✓ | 4 ≥ 3 |
| a4 | 6 | 8 | ✓ | 6 ≥ 6 |
| a5 | 5 | 9 | × | 5 < 8 |
| a6 | 8 | 10 | ✓ | 8 ≥ 8 |

Selected: **{a1, a3, a4, a6}** = 4 activities.

## 08 Solution Q8: Huffman

Each time merge the two smallest: D + B = 6, then 6 + C = 13, then A + E = 22, then 13 + 22

= 35.




| Char | Freq | Code | Bits |
|---|---|---|---|
| A | 10 | 10 | 20 |
| B | 4 | 001 | 12 |
| C | 7 | 01 | 14 |
| D | 2 | 000 | 6 |
| E | 12 | 11 | 24 |

Total = 20 + 12 + 14 + 6 + 24 = **76 bits** (a fixed 3-bit code needs 35 × 3 = 105).